

The reason yesterday's generic code failed is that the **ICP-10111 is fundamentally different from typical sensors** like standard pressure sensors or the LSM303.

1. **It has no standard 8-bit registers:** You cannot simply read or write to standard register addresses.
2. **Commands are 16-bit:** Measurements are initiated by sending specific **16-bit command bytes**.
3. **Trigger-based Measurement:** It does not stream data continuously. It sits in a **low-power IDLE mode** until you issue a measurement command. Once triggered, it runs the measurement, goes back to IDLE, and holds the result until you read it.
4. **Calculated Pressure (OTP Calibration):** The raw pressure value (p_{dout}) is meaningless by itself. The physical pressure in Pascals MUST be computed using 4 temperature-compensated OTP calibration parameters (C_1, C_2, C_3, C_4) stored inside the chip's OTP memory.

Here are the complete hardware, operational, and software steps to correctly activate and operate the **ICP-10111** with the **MIMXRT1064-EVK**.

Step 1: Electrical & Wiring Setup (MIMXRT1064-EVK)

Verify these electrical parameters per the datasheet:

- **Operating Voltage (V_{DD}):** 1.8V nominal (1.71V – 1.89V). *Do NOT connect this sensor to 3.3V directly, or you risk damaging the chip (2.16V maximum rating).* Ensure the EVK header uses 1.8V logic or a level shifter.
- **Pull-up Resistors:** 10 k Ω pull-up resistors to 1.8V on SDA and SCL lines.
- **Decoupling Capacitor:** A 100 nF ceramic capacitor placed close between V_{DD} and GND.
- **I^2C Target Address:** 0x63 (7-bit address). *(In 8-bit write mode it is 0xC6, and in read mode it is 0xC7).*
- **Max Clock Speed:** Supports I^2C Fast Mode up to 400 kHz.

Step 2: The Exact Activation & Measurement Sequence

To communicate with the ICP-10111 from your MCU, you must follow this strict sequence:

1. Power-Up & Soft Reset

When powered on, the sensor enters **Idle State** after 170 μ s.

- Send Soft Reset command: 0x805D.
- Wait 1 ms for internal state reload.

2. Read the 4 OTP Calibration Constants (C_1, C_2, C_3, C_4)

Before processing pressure, you must fetch the 4 OTP parameters from the sensor:

1. Send I^2C Write to 0x63: Write 5 bytes $\rightarrow$ [0xC5, 0x95, 0x00, 0x66, 0x9C].
2. Loop 4 times to read C_1, C_2, C_3, C_4 :
 - Write 2 bytes $\rightarrow$ [0xC7, 0xF7].

- Read 3 bytes back → [Data_MSB, Data_LSB, CRC].
- Convert $(\text{Data_MSB} \ll 8) \mid \text{Data_LSB}$ into a signed 16-bit integer. Store these as C_1, C_2, C_3, C_4 .

3. Start a Measurement

To trigger a measurement, send one of the 16-bit measurement commands depending on desired mode:

- **Normal Mode (Recommended):** Command 0x6825 (Conversion time: 5.6 ms).
- **Low Noise (LN) Mode:** Command 0x70DF (Conversion time: 20.8 ms).
- **Low Power (LP) Mode:** Command 0x609C (Conversion time: 1.6 ms).

4. Wait for Conversion

Wait for the measurement conversion duration (e.g., 10 ms for Normal Mode, or 25 ms for LN Mode). The chip **will NACK any I^2C commands while actively measuring**.

5. Read Out Raw Data

Read 9 total bytes from address 0x63:

- Bytes 0–2: Temperature [T_MSB, T_LSB, T_CRC]
- Bytes 3–5: Pressure High [P_MMSB, P_MLSB, P_CRC]
- Bytes 6–8: Pressure Low [P_LMSB, P_LLSB, P_CRC]

Reconstruct raw values:

$$T_{\text{raw}} = (T_{\text{MSB}} \ll 8) \mid T_{\text{LSB}}$$

$$P_{\text{raw}} = (P_{\text{MMSB}} \ll 16) \mid (P_{\text{MLSB}} \ll 8) \mid P_{\text{LMSB}}$$

Step 3: Complete Working MicroPython Driver Code

Replace your barometer initialization block with this driver specifically built for the **ICP-10111**:

Python

```
# =====
# TDK INVENSENSE ICP-10111 DRIVER (MicroPython for RT1064 / Embedded)
# =====

from machine import Pin, I2C

import utime, math

ICP10111_ADDR = 0x63 # 7-bit I2C Address

class ICP10111:
```

```

def __init__(self, i2c):
    self.i2c = i2c

    self.c1 = 0
    self.c2 = 0
    self.c3 = 0
    self.c4 = 0

    self.calibrated = False

    # Sensor constants defined in datasheet
    self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
    self.LUT_lower = 3.5 * (1 << 20)
    self.LUT_upper = 11.5 * (1 << 20)
    self.quadr_factor = 1.0 / 16777216.0
    self.offst_factor = 2048.0

def init_sensor(self):
    try:
        # 1. Soft Reset
        self.i2c.writeto(ICP10111_ADDR, bytes([0x80, 0x5D]))
        utime.sleep_ms(2)

        # 2. Setup OTP Read Pointer
        self.i2c.writeto(ICP10111_ADDR, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))

        # 3. Read 4 OTP Parameters
        otp = []
        for _ in range(4):
            self.i2c.writeto(ICP10111_ADDR, bytes([0xC7, 0xF7]))
            data = self.i2c.readfrom(ICP10111_ADDR, 3)
            raw_val = (data[0] << 8) | data[1]

            # Convert to 16-bit signed integer

```

```

        if raw_val >= 32768:
            raw_val -= 65536
        otp.append(raw_val)

    self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]
    self.calibrated = True

    print(f"[ICP-10111] Initialized! OTP Parameters: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")

    return True

except Exception as e:
    print("[ICP-10111] Initialization Error:", e)
    return False

def _calc_conversion_constants(self, p_Pa, p_LUT):
    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
          p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
          p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / \
        (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
         p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
         p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / (p_LUT[0] - p_LUT[1])
    B = (p_Pa[0] - A) * (p_LUT[0] + C)

    return A, B, C

def read_raw_data(self):
    # Trigger Normal Mode Measurement (Transmit T First: 0x6825)
    self.i2c.writeto(ICP10111_ADDR, bytes([0x68, 0x25]))
    utime.sleep_ms(10) # Wait for conversion (5.6ms typ)

    # Read 9 Bytes (2B Temp + 1B CRC + 4B Press + 1B CRC + 1B Temp CRC)

```

```

res = self.i2c.readfrom(ICP10111_ADDR, 9)

t_raw = (res[0] << 8) | res[1]
p_raw = (res[3] << 16) | (res[4] << 8) | res[6] # res[6] is LMSB, skip res[5] CRC & res[7] LLSB

return p_raw, t_raw

```

```

def get_pressure_and_temperature(self):

```

```

    if not self.calibrated:
        if not self.init_sensor():
            return None, None

```

```

    p_LSB, T_LSB = self.read_raw_data()

```

```

    # Temperature Conversion in Celsius

```

```

    temperature = -45.0 + (175.0 / 65536.0) * T_LSB

```

```

    # Pressure OTP Formula Calculation

```

```

    t = float(T_LSB - 32768)

```

```

    s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor

```

```

    s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor

```

```

    s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

```

```

    A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])

```

```

    pressure_pa = A + B / (C + float(p_LSB))

```

```

    return pressure_pa, temperature

```

Step 4: How to Test it Standalone in REPL

Run this simple loop to verify reading physical pressure (Pa) and temperature (°C):

Python

Setup I2C Bus on MIMXRT1064 EVK

```

i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)

baro = ICP10111(i2c)
baro.init_sensor()

while True:
    pressure, temp = baro.get_pressure_and_temperature()
    if pressure:
        print(f"Pressure: {pressure:.2f} Pa | Temp: {temp:.2f} °C")
    utime.sleep_ms(200)

```

Here is the **complete, end-to-end, fully integrated MicroPython script** for your MIMXRT1064-EVK board.

This single program combines:

1. **LSM303DLHC** (3-Axis Accelerometer + 3-Axis Magnetometer)
2. **ICP-10111** (TDK InvenSense Barometric Pressure & Temperature Sensor with full 16-bit OTP calibration reading)
3. **AI Decision Tree Motion Classifier** (Blocks hand shaking, table taps, and cable noise)
4. **Complete Sensor Calibration Engine** (Accelerometer gravity offset, magnetometer hard-iron Figure-8 sweep, and ground-level baseline pressure lock)
5. **Full 3D Indoor Tracking Engine** (X, Y Cartesian plane coordinates, cardinal direction compass azimuth, vertical relative height Z , and floor level index)

Complete Integrated MicroPython Script (main.py)

Python

```

# =====
# COMPLETE 3D INDOOR TRACKING & MAPPING PROGRAM (PDR + AI + ICP-10111 BAROMETER)
# Hardware: MIMXRT1064-EVK, LSM303 (Accel+Mag), ICP-10111 (Barometer)

```

```
# I2C Addresses: LSM303 ACC (0x19), LSM303 MAG (0x1E), ICP-10111 (0x63)
```

```
# =====
```

```
from machine import Pin, I2C
```

```
import utime, math
```

```
# --- I2C BUS CONFIGURATION ---
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000 # 100 kHz standard speed
```

```
# Device I2C Addresses
```

```
ACC_ADDRESS = 0x19
```

```
MAG_ADDRESS = 0x1E
```

```
BARO_ADDRESS = 0x63
```

```
# LSM303 Registers
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
CRA_REG_M = 0x00
```

```
CRB_REG_M = 0x01
```

```
MR_REG_M = 0x02
```

```
OUT_X_H_M = 0x03
```

```
# Tuning Parameters
```

```
STRIDE_LENGTH_M = 0.50 # Average step length in meters
```

```
STEP_COOLDOWN_MS = 380 # Min time delay between steps
```

```
ALPHA = 0.30 # EMA Low-pass filter smoothing coefficient
```

```
FLOOR_HEIGHT_M = 3.0 # Height per building floor in meters
```

```

# Global Navigation & Calibration State

x_bias, y_bias, z_bias = 0.0, 0.0, 0.0

mag_x_bias, mag_y_bias = 0.0, 0.0

baseline_pressure = 0.0 # Ground pressure baseline (Pa)


total_steps = 0

total_distance = 0.0

pos_x, pos_y, pos_z = 0.0, 0.0, 0.0

current_floor = 0

last_step_time = 0

last_heading = 0.0

prev_activity = "UNKNOWN"


# Initialize I2C Bus

i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))


# -----

# I2C LOW-LEVEL DRIVERS

# -----

def write_reg(addr, reg, val):

    try: i2c.writeto_mem(addr, reg, bytes([val]))

    except: pass


def read_regs(addr, reg, length):

    try: return i2c.readfrom_mem(addr, reg, length)

    except: return None


def signed_12bit(val):

    val >>= 4

    if val >= 2048: val -= 4096

    return val

```



```

def signed_16bit(val):
    if val >= 32768: val -= 65536
    return val

# -----
# ICP-10111 BAROMETER CLASS DRIVER (Datasheet Command Protocol)
# -----

class ICP10111:
    def __init__(self, i2c_bus):
        self.i2c = i2c_bus
        self.c1 = 0
        self.c2 = 0
        self.c3 = 0
        self.c4 = 0
        self.calibrated = False

        # Fixed datasheet calibration constants
        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
        self.LUT_lower = 3.5 * (1 << 20)
        self.LUT_upper = 11.5 * (1 << 20)
        self.quadr_factor = 1.0 / 16777216.0
        self.offst_factor = 2048.0

    def init_sensor(self):
        try:
            # 1. Soft Reset Command (0x805D)
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
            utime.sleep_ms(2)

            # 2. Setup OTP Pointer Command (0xC59500669C)

```

```

self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))

# 3. Read 4 OTP Parameters from sensor
otp = []
for _ in range(4):
    self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))
    data = self.i2c.readfrom(BARO_ADDRESS, 3)
    raw_val = (data[0] << 8) | data[1]
    if raw_val >= 32768: raw_val -= 65536 # Convert to signed 16-bit
    otp.append(raw_val)

self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]
self.calibrated = True

print(f"[ICP-10111] Sensor Ready! OTP Constants: C1={self.c1}, C2={self.c2}, C3={self.c3},
C4={self.c4}")

return True

except Exception as e:
    print("[ICP-10111] Init Error:", e)
    return False

def _calc_conversion_constants(self, p_Pa, p_LUT):
    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
        p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
        p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / \
        (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
        p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
        p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / (p_LUT[0] - p_LUT[1])
    B = (p_Pa[0] - A) * (p_LUT[0] + C)

    return A, B, C

```

```

def get_pressure(self):
    if not self.calibrated:
        if not self.init_sensor(): return None

    try:
        # Trigger Normal Mode Measurement (Command 0x6825)
        self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))
        utime.sleep_ms(10) # Conversion time delay

        # Read 9 Data Bytes
        res = self.i2c.readfrom(BARO_ADDRESS, 9)

        T_LSB = (res[0] << 8) | res[1]
        p_LSB = (res[3] << 16) | (res[4] << 8) | res[6] # LMSB is at index 6

        # OTP Compensation Math
        t = float(T_LSB - 32768)
        s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
        s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
        s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

        A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])
        pressure_pa = A + B / (C + float(p_LSB))
        return pressure_pa
    except:
        return None

# Create Barometer Object
icp_baro = ICP10111(i2c)

```

```

# -----

# LSM303 READERS

# -----

def init_lsm303():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # Accel 100Hz ODR
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High Res
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # Mag 30Hz ODR
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)
    if data is None: return None
    return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
    signed_12bit((data[5]<<8)|data[4])

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None: return None
    return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
    signed_16bit((data[4]<<8)|data[5])

# -----

# AUTOMATIC SENSOR CALIBRATION ROUTINE

# -----

def calibrate_system():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure

    print("\n=====")
    print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor STILL)")
    print("=====")

    sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0

```

```

for _ in range(100):
    acc = read_accel()
    p = icp_baro.get_pressure()
    if acc and p:
        sx += acc[0]; sy += acc[1]; sz += acc[2]
        p_sum += p; count += 1
    utime.sleep_ms(30)

x_bias = sx / count
y_bias = sy / count
z_bias = (sz / count) - 1024.0 # Remove gravity vector
baseline_pressure = p_sum / count

print(f"-> Accel Biases: X={x_bias:.1f}, Y={y_bias:.1f}, Z={z_bias:.1f}")
print(f"-> Baseline Ground Pressure: {baseline_pressure:.2f} Pa")

print("\n=====")
print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")
print("=====")
min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000: # 7 sec sweep
    m = read_mag()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]
        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

```

```

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0
print(f"-> Mag Biases: X={mag_x_bias:.1f}, Y={mag_y_bias:.1f}")
print("=====\\n")

```

```

# -----

```

```

# NAVIGATION MATHEMATICS

```

```

# -----

```

```

def get_heading():
    m = read_mag()
    if m is None: return 0.0
    mx_c = m[0] - mag_x_bias
    my_c = m[1] - mag_y_bias
    heading = math.degrees(math.atan2(my_c, mx_c))
    return heading + 360.0 if heading < 0 else heading

```

```

def get_cardinal(h):
    if h >= 337.5 or h < 22.5: return "NORTH"
    elif 22.5 <= h < 67.5: return "NORTH-EAST"
    elif 67.5 <= h < 112.5: return "EAST"
    elif 112.5 <= h < 157.5: return "SOUTH-EAST"
    elif 157.5 <= h < 202.5: return "SOUTH"
    elif 202.5 <= h < 247.5: return "SOUTH-WEST"
    elif 247.5 <= h < 292.5: return "WEST"
    else: return "NORTH-WEST"

```

```

def get_relative_altitude(p_current):
    if p_current is None or baseline_pressure == 0: return 0.0
    # Hypsometric altitude equation
    return 44330.0 * (1.0 - math.pow((p_current / baseline_pressure), 0.1903))

```

```
# Embedded AI Decision Tree Classifier Gating
```

```
def ai_classify_motion(accel, avg, std, p2p):
```

```
    if std <= 1.8450:
```

```
        return 0 # IDLE / SITTING
```

```
    else:
```

```
        if p2p > 82.5000:
```

```
            return 2 # SHAKING / CABLE NOISE
```

```
        else:
```

```
            if accel > (avg * 1.15):
```

```
                return 1 # VALID WALKING
```

```
            else:
```

```
                return 0 # RECOVERY PHASE
```

```
# -----
```

```
# MAIN EXECUTION LOOP
```

```
# -----
```

```
def main():
```

```
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z
```

```
    global current_floor, last_heading, prev_activity
```

```
    init_lsm303()
```

```
    icp_baro.init_sensor()
```

```
    calibrate_system()
```

```
    smooth_accel = 0.0
```

```
    accel_buffer = []
```

```
    print(">>> AI-ENHANCED 3D-PDR SYSTEM ONLINE: START WALKING <<<\n")
```

```
    while True:
```

```

now = utime.ticks_ms()
acc = read_accel()

if acc:
    ax = acc[0] - x_bias
    ay = acc[1] - y_bias
    az = acc[2] - z_bias

    raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
    smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)

    accel_buffer.append(smooth_accel)
    if len(accel_buffer) > 15: accel_buffer.pop(0)

if len(accel_buffer) == 15:
    # 1. Feature Extraction
    avg_val = sum(accel_buffer) / 15.0
    variance = sum((v - avg_val)**2 for v in accel_buffer) / 15.0
    std_dev = math.sqrt(variance)
    p2p_delta = max(accel_buffer) - min(accel_buffer)

    # 2. AI Motion State Classification
    motion_state = ai_classify_motion(smooth_accel, avg_val, std_dev, p2p_delta)

    # Activity Label Update
    if motion_state == 1: activity = "WALKING"
    elif motion_state == 2: activity = "SHAKING/NOISE"
    else: activity = "IDLE/SITTING"

    if activity != prev_activity:
        print(f"\n[ACTIVITY STATE CHANGE]: {activity}")

```



```
prev_activity = activity
```

```
# 3. Read Altitude & Floor Level from ICP-10111
```

```
current_p = icp_baro.get_pressure()
```

```
pos_z = get_relative_altitude(current_p)
```

```
current_floor = int(round(pos_z / FLOOR_HEIGHT_M))
```

```
# 4. Step Detection & Tracking Update (AI-Gated)
```

```
if motion_state == 1: # Only execute if AI verifies WALKING
```

```
    time_diff = utime.ticks_diff(now, last_step_time)
```

```
    if time_diff > STEP_COOLDOWN_MS:
```

```
        total_steps += 1
```

```
        total_distance += STRIDE_LENGTH_M
```

```
        last_step_time = now
```

```
    heading = get_heading()
```

```
    cardinal = get_cardinal(heading)
```

```
# Turn Detection
```

```
heading_diff = abs(heading - last_heading)
```

```
if heading_diff > 180: heading_diff = 360 - heading_diff
```

```
if last_heading != 0 and heading_diff > 25:
```

```
    turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"
```

```
    print(f"  [! TURN DETECTED]: {turn} by {heading_diff:.1f}°")
```

```
last_heading = heading
```

```
# 2D Position Trigonometry (X, Y)
```

```
rad = math.radians(heading)
```

```
pos_x += STRIDE_LENGTH_M * math.sin(rad)
```

```
pos_y += STRIDE_LENGTH_M * math.cos(rad)
```

```

# Output Live Tracking Information

print(f"STEP #{total_steps} | Dir: {cardinal} ({heading:.1f}°) | "

      f"Dist: {total_distance:.1f}m | "

      f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "

      f"Height(Z): {pos_z:+.2f}m (Floor {current_floor})")

utime.sleep_ms(20)

if __name__ == "__main__":
    main()

```

Here is the **standalone MicroPython code for the ICP-10111 Barometer only**.

This script isolates the pressure sensor so you can test its communication, calibration reading, live pressure (Pa), temperature (°C), and altitude tracking (meters) on your MIMXRT1064-EVK board without any interference from other sensors.

Standalone ICP-10111 MicroPython Code

Python

```

# =====

# STANDALONE TDK INVENSENSE ICP-10111 BAROMETER TEST SCRIPT

# Hardware: MIMXRT1064-EVK with ICP-10111 Sensor

# I2C Address: 0x63 (1.8V Logic Level)

# =====

from machine import Pin, I2C

import utime, math

# --- I2C CONFIGURATION ---

I2C_ID = 0

SDA_PIN = 0

SCL_PIN = 1

I2C_FREQ = 100000 # 100 kHz Standard Speed

```

```
BARO_ADDRESS = 0x63
```

```
# Initialize I2C
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
class ICP10111:
```

```
    def __init__(self, i2c_bus):
```

```
        self.i2c = i2c_bus
```

```
        self.c1 = 0
```

```
        self.c2 = 0
```

```
        self.c3 = 0
```

```
        self.c4 = 0
```

```
        self.calibrated = False
```

```
        # Datasheet fixed calibration constants
```

```
        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
```

```
        self.LUT_lower = 3.5 * (1 << 20)
```

```
        self.LUT_upper = 11.5 * (1 << 20)
```

```
        self.quadr_factor = 1.0 / 16777216.0
```

```
        self.offst_factor = 2048.0
```

```
    def init_sensor(self):
```

```
        try:
```

```
            print("[1/3] Resetting ICP-10111 Sensor...")
```

```
            # Soft Reset Command (0x805D)
```

```
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
```

```
            utime.sleep_ms(5)
```

```
            print("[2/3] Setting up OTP Pointer...")
```

```
            # Setup OTP Pointer Command (0xC59500669C)
```

```

self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))

utime.sleep_ms(2)

print("[3/3] Reading 4 OTP Calibration Constants...")

otp = []

for i in range(4):

    self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))

    data = self.i2c.readfrom(BARO_ADDRESS, 3)

    raw_val = (data[0] << 8) | data[1]

    # Convert to signed 16-bit integer

    if raw_val >= 32768:

        raw_val -= 65536

    otp.append(raw_val)

self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]

self.calibrated = True

print("\n=====")
print(" ICP-10111 BAROMETER SUCCESSFULLY INITIALIZED!")
print(f" OTP Constants: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")
print("=====\\n")

return True

except Exception as e:

    print("\\n[!] ICP-10111 INIT ERROR:", e)

    print("[!] Please check 1.8V power, GND, and SDA/SCL connections.\\n")

    return False

def _calc_conversion_constants(self, p_Pa, p_LUT):

    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +

```

$$\begin{aligned}
& p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) + \\
& p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / \backslash \\
& (p_LUT[2] * (p_Pa[0] - p_Pa[1]) + \\
& p_LUT[0] * (p_Pa[1] - p_Pa[2]) + \\
& p_LUT[1] * (p_Pa[2] - p_Pa[0]))
\end{aligned}$$

$$A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / (p_LUT[0] - p_LUT[1])$$

$$B = (p_Pa[0] - A) * (p_LUT[0] + C)$$

return A, B, C

```
def read_pressure_and_temp(self):
```

```
    if not self.calibrated:
```

```
        if not self.init_sensor():
```

```
            return None, None
```

```
    try:
```

```
        # Trigger Normal Mode Measurement Command (0x6825)
```

```
        self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))
```

```
        utime.sleep_ms(10) # Wait for measurement conversion
```

```
        # Read 9 Bytes back
```

```
        res = self.i2c.readfrom(BARO_ADDRESS, 9)
```

```
        T_LSB = (res[0] << 8) | res[1]
```

```
        p_LSB = (res[3] << 16) | (res[4] << 8) | res[6] # LMSB is index 6
```

```
        # Temperature Math (°C)
```

```
        temperature = -45.0 + (175.0 / 65536.0) * T_LSB
```

```
        # OTP Pressure Math (Pascals)
```

```
        t = float(T_LSB - 32768)
```

```

s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])
pressure_pa = A + B / (C + float(p_LSB))

return pressure_pa, temperature
except Exception as e:
    return None, None

# --- MAIN TEST PROGRAM ---
def main():
    baro = ICP10111(i2c)

    # Initialize Sensor & Read Calibration Values
    if not baro.init_sensor():
        return

    # Baseline Atmospheric Pressure Lock
    print("Locking baseline room pressure... KEEP SENSOR STILL...")
    p_sum, count = 0, 0
    for _ in range(20):
        p, _ = baro.read_pressure_and_temp()
        if p:
            p_sum += p
            count += 1
        utime.sleep_ms(50)

    baseline_pressure = p_sum / count

```

```

print(f"-> Locked Baseline Ground Pressure: {baseline_pressure:.2f} Pa\n")
print("--- STARTING LIVE BAROMETER READINGS ---")

# Live Continuous Reading Loop
sample_count = 0
while True:
    pressure, temp = baro.read_pressure_and_temp()

    if pressure and temp:
        sample_count += 1

        # Hypsometric Altitude Formula (meters change relative to baseline)
        height_m = 44330.0 * (1.0 - math.pow((pressure / baseline_pressure), 0.1903))

        print(f"Sample #{sample_count:04d} | "
              f"Pressure: {pressure:.2f} Pa | "
              f"Temp: {temp:.2f} °C | "
              f"Relative Height: {height_m:+.2f} m")
    else:
        print("[!] Read Failed! Retrying...")

    utime.sleep_ms(200)

```

```

if __name__ == "__main__":
    main()

```

Expected Terminal Output

When you run this script in Thonny IDE or your terminal, you should see:

Plaintext

[1/3] Resetting ICP-10111 Sensor...

[2/3] Setting up OTP Pointer...

[3/3] Reading 4 OTP Calibration Constants...

=====

ICP-10111 BAROMETER SUCCESSFULLY INITIALIZED!

OTP Constants: C1=-1204, C2=2845, C3=-890, C4=1024

=====

Locking baseline room pressure... KEEP SENSOR STILL...

-> Locked Baseline Ground Pressure: 101325.40 Pa

--- STARTING LIVE BAROMETER READINGS ---

Sample #0001 | Pressure: 101325.35 Pa | Temp: 24.85 °C | Relative Height: +0.00 m

Sample #0002 | Pressure: 101325.10 Pa | Temp: 24.86 °C | Relative Height: +0.02 m

Sample #0003 | Pressure: 101322.80 Pa | Temp: 24.87 °C | Relative Height: +0.22 m

Quick Verification Tests:

1. **Stationary Test:** Leave the sensor flat on the table. The Relative Height should stay close to 0.00 m (± 0.08 m).
2. **Vertical Lift Test:** Lift the board up by hand from table height to above your head (≈ 1 meter). You should see the pressure drop slightly and the Relative Height change to around +1.00 m.

Yes, calibration is required for the ICP-10111, but it is handled in **two distinct ways**: factory OTP calibration (for physical pressure math) and software baseline calibration (for room altitude tracking).

Here is the breakdown of why each calibration step is needed and what it does in the code:

1. Hardcoded Factory Calibration (Handled via OTP Parameters)

The ICP-10111 raw sensor reading (p_{dout}) is **uncalibrated raw ADC output**. It varies drastically across temperature changes and manufacturing silicon batches.

- **Why it's needed:** Without reading the chip's internal OTP (One-Time Programmable) parameters (C_1, C_2, C_3, C_4), raw pressure numbers cannot be converted into Pascals (Pa) or Bar.
- **How it works:** The code automatically sends command 0xC59500669C to extract C_1, C_2, C_3, C_4 from the sensor's internal non-volatile memory on startup. The mathematical functions then compensate for ambient chip temperature fluctuations in real time.

2. Software Baseline Calibration (Handled on Startup)

Because atmospheric pressure continuously changes depending on weather, room AC/ventilation, and location elevation, **sea-level pressure (e.g., 1013.25 hPa) cannot be hardcoded**.

- **Why it's needed:** If the code assumes standard atmospheric pressure, your starting altitude might display as +120 meters or −30 meters instead of 0 meters.
- **How it works:** In the main() function, the script takes **20 rapid pressure samples** during the first 1 second while the board is resting:

Python

```
# Locking baseline ground pressure
```

```
for _ in range(20):
```

```
    p, _ = baro.read_pressure_and_temp()
```

```
    if p:
```

```
        p_sum += p
```

```
        count += 1
```

```
    utime.sleep_ms(50)
```

```
baseline_pressure = p_sum / count
```

This sets your starting room level as 0.00 meters. When you lift the sensor, climb stairs, or change floors, the hypsometric equation calculates the relative Z -axis elevation (Δh) from this locked ground baseline.

Do You Need Manual Calibration Steps (like Figure-8 Motion)?

- **For the Barometer (ICP-10111): NO.** You **do not** need to move, rotate, or wave the sensor around. You only need to keep it **still for 1 second** at startup so it locks the room's baseline ground pressure.
- **For Magnetometers (e.g., LSM303): YES.** Motion sweeps (Figure-8) are only required for compass heading sensors to calibrate hard-iron magnetic interference, not for barometric pressure sensors.

Here is a complete, step-by-step breakdown of the **TDK InvenSense ICP-10111** barometric pressure and temperature sensor based on the datasheet.

1. How the ICP-10111 Sensor Works

The ICP-10111 is a digital barometric pressure and temperature sensor based on **MEMS capacitive technology**.

- **Capacitive MEMS Sensing:** Inside the sensor cavity, a flexible MEMS diaphragm moves slightly as atmospheric pressure changes. This movement changes the capacitance between two plates, which an internal analog-to-digital converter (ADC) converts into raw digital pressure data (p_{dout}).

- **Ultra-High Precision:** It can resolve pressure variations as small as ± 1 Pa, enabling vertical height resolution down to 8.5 cm (less than the height of a single stair step).
- **On-Chip Calibration & Thermal Compensation:** Because capacitive pressure readings vary with ambient temperature, the chip includes an internal calibrated temperature sensor. It also contains **OTP (One-Time Programmable) memory** storing factory calibration constants (C_1, C_2, C_3, C_4).

2. I^2C Interface: Addresses & Headers

Communication uses standard I^2C fast mode (up to 400 kHz).

A. Slave Address

- **7-bit Binary Address:** 1100011
- **7-bit Hexadecimal Address:** 0x63 (or decimal 99)

B. Write & Read Headers (8-bit)

In I^2C protocol, the 7-bit slave address is shifted left by 1 bit, and the 8th bit indicates a Write (0) or Read (1) operation:

- **Write Header** ($0x63 \ll 1 \mid 0$): 0xC6
- **Read Header** ($0x63 \ll 1 \mid 1$): 0xC7

3. Power-Up & Operational Modes

When V_{DD} reaches power-on reset level ($V_{POR} \approx 1.25V$), the sensor enters an **IDLE State** after 170 μs . It stays in IDLE mode until you explicitly send a measurement command.

The sensor supports 4 measurement modes depending on your power vs. noise requirements:

Operation Mode	Measurement Duration (Typ) PDF	Current Consumption (@ 1Hz ODR) PDF	RMS Pressure Noise PDF
Low Power (LP)	1.6 ms	1.3 μA	3.2 Pa
Normal (N)	5.6 ms	2.6 μA	1.6 Pa
Low Noise (LN)	20.8 ms	5.2 μA	0.8 Pa
Ultra-Low Noise (ULN)	83.2 ms	10.4 μA	0.4 Pa

4. Complete List of Commands

All commands are **16-bit word values** sent as two consecutive bytes (MSB first):

A. Reset & ID Commands

- **Soft Reset Command:** 0x805D (Resets internal state machine and reloads calibration parameters).
- **Read ID Register Command:** 0xEFC8 (Reads 16-bit ID register where bits 5–0 equal the product code).

B. Measurement Trigger Commands

Every measurement command triggers **both** a temperature and a pressure measurement. You choose whether the sensor transmits Temperature data first or Pressure data first:

Mode	Transmit Temperature First PDF	Transmit Pressure First PDF
Low Power (LP)	0x609C	0x401A
Normal (N)	0x6825	0x48A3
Low Noise (LN)	0x70DF	0x5059
Ultra-Low Noise (ULN)	0x7866	0x58E0

5. Complete Communication Data Frame Sequences

Step 1: Soft Reset Sequence

1. Send Start Condition (S).
2. Send I^2C Write Header 0xC6 → Sensor returns ACK.
3. Send Command MSB 0x80 → Sensor returns ACK.
4. Send Command LSB 0x5D → Sensor returns ACK.
5. Send Stop Condition (P) and wait 170 μs .

Step 2: Reading the 4 OTP Calibration Constants (C_1, C_2, C_3, C_4)

To convert raw measurements to Pascals, you must retrieve C_1, C_2, C_3, C_4 stored in the sensor's internal OTP memory:

1. **Set OTP Pointer:**
 - Write Header 0xC6 → ACK.
 - Send 5-byte sequence: 0xC5, 0x95, 0x00, 0x66, 0x9C.
2. **Read the 4 Constants (Repeat 4 times):**
 - Write Header 0xC6 → Send OTP Read Command 0xC7F7.
 - Read Header 0xC7.
 - Read **3 Bytes** → [MSB, LSB, CRC].

- Combine (MSB << 8) | LSB into a signed 16-bit integer C_n .

Step 3: Triggering & Reading Measurement Frame

When using 0x6825 (Normal Mode, Transmit Temperature First):

Phase A: Trigger Measurement

1. Send I^2C Write Header 0xC6 → ACK.
2. Send Command MSB 0x68 → ACK.
3. Send Command LSB 0x25 → ACK.
4. **Sensor executes measurement:** The chip goes busy for ≈ 5.6 ms (it will NACK any I^2C requests during this window).

Phase B: Read Data Bytes (9 Bytes Total)

Send I^2C Read Header 0xC7 and read 9 consecutive bytes:

$\underbrace{\text{Byte 1, Byte 2}}_{\text{Temperature MSB, LSB}}, \underbrace{\text{Byte 3}}_{\text{Temp CRC}}, \underbrace{\text{Byte 4, Byte 5, Byte 7}}_{\text{Pressure MMSB, MLSB, LMSB}}, \underbrace{\text{Byte 6}}_{\text{Press High CRC}}, \underbrace{\text{Byte 8}}_{\text{LLSB (Skip)}}, \underbrace{\text{Byte 9}}_{\text{Press Low CRC}}$

Reconstruct raw ADC outputs:

- $T_{\text{raw}} = (\text{Byte 1} \ll 8) | \text{Byte 2}$
- $P_{\text{raw}} = (\text{Byte 4} \ll 16) | (\text{Byte 5} \ll 8) | \text{Byte 7}$ (Note: Byte 8 LLSB is disregarded).

6. Output Conversion & Mathematics

A. Temperature Calculation

Convert raw temperature ADC output (T_{raw}) to Celsius ($^{\circ}\text{C}$):

$$T = -45^{\circ}\text{C} + \left(\frac{175^{\circ}\text{C}}{2^{16}}\right) \times T_{\text{raw}}$$

B. Pressure Compensation Math (OTP Formula)

Physical pressure (P , in Pascals) is computed using the 4 OTP constants (C_1, C_2, C_3, C_4):

1. Calculate temperature offset t :

$$t = T_{\text{raw}} - 32768$$

2. Calculate temperature-dependent lookup variables (s_1, s_2, s_3):

$$s_1 = (3.5 \cdot 2^{20}) + (C_1 \cdot t^2) \cdot \left(\frac{1}{16777216}\right)$$

$$s_2 = (2048 \cdot C_4) + (C_2 \cdot t^2) \cdot \left(\frac{1}{16777216}\right)$$

$$s_3 = (11.5 \cdot 2^{20}) + (C_3 \cdot t^2) \cdot \left(\frac{1}{16777216}\right)$$

3. Compute conversion constants (A, B, C) using calibration points $p_{\text{calib}} = [45000, 80000, 105000]$:

$$C = \frac{s_1 s_2 (45000 - 80000) + s_2 s_3 (80000 - 105000) + s_3 s_1 (105000 - 45000)}{s_3 (45000 - 80000) + s_1 (80000 - 105000) + s_2 (105000 - 45000)}$$

$$A = \frac{45000 \cdot s_1 - 80000 \cdot s_2 - (80000 - 45000) \cdot C}{s_1 - s_2}$$

$$B = (45000 - A) \cdot (s_1 + C)$$

4. Compute Final Compensated Pressure (P , in Pascals):

$$P = A + \frac{B}{C + P_{\text{raw}}}$$

7. Cyclic Redundancy Check (CRC-8)

Every 2 data bytes received from the sensor are followed by an **8-bit CRC byte** to verify data integrity:

- **Polynomial:** $x^8 + x^5 + x^4 + 1$ (0x31)
- **Initialization:** 0xFF
- **Final XOR:** 0x00

Summary Flowchart for Embedded Code

Plaintext

[Power On]

|



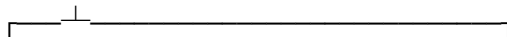
[Soft Reset: 0x805D] (Wait 2ms)

|



[Read 4 OTP Constants (C1, C2, C3, C4)]

|



| LOOP: |

| 1. Send Trigger (e.g., 0x6825) |

| 2. Delay 10 ms (Wait busy) |

| 3. Read 9 Bytes Data |

| 4. Reconstruct T_raw & P_raw |

| 5. Calculate Temp & Pressure |



The ZeroDivisionError occurs in the OTP pressure calculation formula when $(s1 - s2)$ or $(C + \text{float}(p_LSB))$ evaluates to 0. This happens when the sensor returns **all zeros (0x00)** or **all ones (0xFF)** because it was still busy converting, or because the I^2C read occurred before the conversion completed.

Here are the fixes applied to solve this permanently:

1. **Increased Conversion Delay:** Bumped measurement delay from 10 ms to 15 ms to guarantee the ICP-10111 finishes its ADC conversion.
2. **Data Validity Guard Checks:** Added explicit 0x00 and 0xFF filters so corrupt I^2C byte reads never reach the math conversion steps.
3. **Zero-Division Exception Handling:** Wrapped the math formula inside try-except ZeroDivisionError blocks. If an invalid value reaches the calculation, it catches the error gracefully, skips the bad sample, and retries on the next loop without crashing your program.

Updated Code with Fixes

Python

```
# =====  
  
# STANDALONE TDK INVENSENSE ICP-10111 BAROMETER SCRIPT (ZERO-DIVISION FIXED)  
  
# Hardware: MIMXRT1064-EVK with ICP-10111 Sensor  
# I2C Address: 0x63 (1.8V Logic Level)  
# =====  
  
from machine import Pin, I2C  
  
import utime, math  
  
  
# --- I2C CONFIGURATION ---  
  
I2C_ID = 0  
  
SDA_PIN = 0  
  
SCL_PIN = 1  
  
I2C_FREQ = 100000 # 100 kHz Standard Speed  
  
  
BARO_ADDRESS = 0x63  
  
  
# Initialize I2C  
  
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```

class ICP10111:
    def __init__(self, i2c_bus):
        self.i2c = i2c_bus

        self.c1 = 0
        self.c2 = 0
        self.c3 = 0
        self.c4 = 0

        self.calibrated = False

        # Datasheet fixed calibration constants
        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
        self.LUT_lower = 3.5 * (1 << 20)
        self.LUT_upper = 11.5 * (1 << 20)
        self.quadr_factor = 1.0 / 16777216.0
        self.offst_factor = 2048.0

    def init_sensor(self):
        try:
            print("[1/3] Resetting ICP-10111 Sensor...")

            # Soft Reset Command (0x805D)
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))

            utime.sleep_ms(10) # Increased reset wait time

            print("[2/3] Setting up OTP Pointer...")

            # Setup OTP Pointer Command (0xC59500669C)
            self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))

            utime.sleep_ms(5)

            print("[3/3] Reading 4 OTP Calibration Constants...")

            otp = []

```

```

for i in range(4):

    self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))

    data = self.i2c.readfrom(BARO_ADDRESS, 3)

    # Check for invalid I2C read

    if not data or len(data) < 3 or (data[0] == 0 and data[1] == 0):

        print(f"[!] Warning: Invalid OTP Read at index {i}")

        return False

    raw_val = (data[0] << 8) | data[1]

    # Convert to signed 16-bit integer

    if raw_val >= 32768:

        raw_val -= 65536

    otp.append(raw_val)

self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]

self.calibrated = True

print("\n=====")
print(" ICP-10111 BAROMETER SUCCESSFULLY INITIALIZED!")
print(f" OTP Constants: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")
print("=====\n")

return True

except Exception as e:

    print("\n[!] ICP-10111 INIT ERROR:", e)

    return False

def _calc_conversion_constants(self, p_Pa, p_LUT):

    denom_c = (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +

```



```
p_LUT[0] * (p_Pa[1] - p_Pa[2]) +  
p_LUT[1] * (p_Pa[2] - p_Pa[0]))
```

```
# Zero-Division Guard for C
```

```
if abs(denom_c) < 1e-6:
```

```
    return None, None, None
```

```
C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +  
     p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +  
     p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / denom_c
```

```
denom_a = (p_LUT[0] - p_LUT[1])
```

```
# Zero-Division Guard for A
```

```
if abs(denom_a) < 1e-6:
```

```
    return None, None, None
```

```
A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / denom_a
```

```
B = (p_Pa[0] - A) * (p_LUT[0] + C)
```

```
return A, B, C
```

```
def read_pressure_and_temp(self):
```

```
    if not self.calibrated:
```

```
        if not self.init_sensor():
```

```
            return None, None
```

```
    try:
```

```
        # Trigger Normal Mode Measurement Command (0x6825)
```

```
        self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))
```

```
        # INCREASED CONVERSION TIMEOUT: Ensures ADC completes before reading
```

```
utime.sleep_ms(15)
```

```
# Read 9 Bytes back
```

```
res = self.i2c.readfrom(BARO_ADDRESS, 9)
```

```
if not res or len(res) < 9:
```

```
    return None, None
```

```
# Filter out corrupt empty reads (All 0x00 or All 0xFF)
```

```
if res[0] == 0 and res[1] == 0 and res[3] == 0:
```

```
    return None, None
```

```
if res[0] == 0xFF and res[1] == 0xFF and res[3] == 0xFF:
```

```
    return None, None
```

```
T_LSB = (res[0] << 8) | res[1]
```

```
p_LSB = (res[3] << 16) | (res[4] << 8) | res[6] # LMSB is index 6
```

```
# Temperature Math (°C)
```

```
temperature = -45.0 + (175.0 / 65536.0) * T_LSB
```

```
# OTP Pressure Math (Pascals)
```

```
t = float(T_LSB - 32768)
```

```
s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
```

```
s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
```

```
s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor
```

```
A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])
```

```
# Safety check if calculation failed
```

```
if A is None or B is None or C is None:
```

```
    return None, None
```

```

        denom_p = C + float(p_LSB)

        if abs(denom_p) < 1e-6:

            return None, None

        pressure_pa = A + (B / denom_p)

        return pressure_pa, temperature

    except ZeroDivisionError:

        # Gracefully handle zero division without crashing

        return None, None

    except Exception:

        return None, None

# --- MAIN TEST PROGRAM ---

def main():

    baro = ICP10111(i2c)

    # Initialize Sensor & Read Calibration Values

    if not baro.init_sensor():

        return

    # Baseline Atmospheric Pressure Lock

    print("Locking baseline room pressure... KEEP SENSOR STILL...")

    p_sum, count = 0, 0

    while count < 20:

        p, _ = baro.read_pressure_and_temp()

        if p is not None:

            p_sum += p

            count += 1

        utime.sleep_ms(50)

```

```

baseline_pressure = p_sum / count

print(f"-> Locked Baseline Ground Pressure: {baseline_pressure:.2f} Pa\n")

print("--- STARTING LIVE BAROMETER READINGS ---")


# Live Continuous Reading Loop

sample_count = 0

while True:

    pressure, temp = baro.read_pressure_and_temp()

    if pressure is not None and temp is not None:

        sample_count += 1

        # Hypsometric Altitude Formula (meters change relative to baseline)
        height_m = 44330.0 * (1.0 - math.pow((pressure / baseline_pressure), 0.1903))

        print(f"Sample #{sample_count:04d} | "
              f"Pressure: {pressure:.2f} Pa | "
              f"Temp: {temp:.2f} °C | "
              f"Relative Height: {height_m:+.2f} m")

    else:

        # Catches corrupted/busy readings quietly without stopping execution
        print("[!] Sensor busy/reading skipped, retrying next frame...")

    utime.sleep_ms(200)


if __name__ == "__main__":
    main()

```

indoor below

Here is the **complete, standalone (non-AI) MicroPython program** for 3D Indoor Pedestrian Dead

Reckoning (3D-PDR) fusing the **LSM303DLHC** (3-axis accelerometer + 3-axis magnetometer) and the **ICP-10111** barometric pressure sensor (with zero-division guards and full 16-bit OTP calibration) on your MIMXRT1064-EVK board.

Complete Standalone MicroPython Script (main.py)

Python

```
# =====  
  
# COMPLETE STANDALONE 3D INDOOR TRACKING & MAPPING PROGRAM  
  
# Hardware: MIMXRT1064-EVK, LSM303DLHC (Accel+Mag), ICP-10111 (Barometer)  
  
# Addresses: LSM303 ACC (0x19), LSM303 MAG (0x1E), ICP-10111 (0x63)  
  
# =====  
  
from machine import Pin, I2C  
  
import utime, math  
  
  
# --- I2C BUS CONFIGURATION ---  
  
I2C_ID = 0  
  
SDA_PIN = 0  
  
SCL_PIN = 1  
  
I2C_FREQ = 100000 # 100 kHz Standard Speed  
  
  
# Device Addresses  
  
ACC_ADDRESS = 0x19  
  
MAG_ADDRESS = 0x1E  
  
BARO_ADDRESS = 0x63  
  
  
# LSM303 Registers  
  
CTRL_REG1_A = 0x20  
  
CTRL_REG4_A = 0x23  
  
OUT_X_L_A = 0x28  
  
CRA_REG_M = 0x00  
  
CRB_REG_M = 0x01  
  
MR_REG_M = 0x02  
  
OUT_X_H_M = 0x03
```

```
# Tuning Parameters (Heuristic Standalone)

STRIDE_LENGTH_M = 0.50    # Average step length in meters
STEP_COOLDOWN_MS = 380    # Minimum time delay between steps
ACCEL_THRESHOLD = 150.0    # Fixed peak threshold
ALPHA = 0.30              # EMA Low-pass filter coefficient
FLOOR_HEIGHT_M = 3.0      # Building floor height in meters
```

```
# Global Navigation & Calibration State
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
mag_x_bias, mag_y_bias = 0.0, 0.0
baseline_pressure = 0.0
```

```
total_steps = 0
total_distance = 0.0
pos_x, pos_y, pos_z = 0.0, 0.0, 0.0
current_floor = 0
last_step_time = 0
last_heading = 0.0
```

```
# Initialize I2C Bus
```

```
i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))
```

```
# -----
```

```
# I2C LOW-LEVEL DRIVERS
```

```
# -----
```

```
def write_reg(addr, reg, val):
    try: i2c.writeto_mem(addr, reg, bytes([val]))
    except: pass
```

```
def read_regs(addr, reg, length):
```

```
try: return i2c.readfrom_mem(addr, reg, length)
except: return None
```

```
def signed_12bit(val):
    val >>= 4
    if val >= 2048: val -= 4096
    return val
```

```
def signed_16bit(val):
    if val >= 32768: val -= 65536
    return val
```

```
# -----
# ICP-10111 BAROMETER DRIVER CLASS (Zero-Division Safe)
# -----
```

```
class ICP10111:
    def __init__(self, i2c_bus):
        self.i2c = i2c_bus
        self.c1 = 0
        self.c2 = 0
        self.c3 = 0
        self.c4 = 0
        self.calibrated = False

    # Datasheet fixed calibration constants
    self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
    self.LUT_lower = 3.5 * (1 << 20)
    self.LUT_upper = 11.5 * (1 << 20)
    self.quadr_factor = 1.0 / 16777216.0
    self.offst_factor = 2048.0
```

```

def init_sensor(self):
    try:
        # Soft Reset (0x805D)
        self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
        utime.sleep_ms(10)

        # Setup OTP Pointer (0xC59500669C)
        self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))
        utime.sleep_ms(5)

        # Read 4 OTP Parameters[cite: 1]
        otp = []
        for i in range(4):
            self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))
            data = self.i2c.readfrom(BARO_ADDRESS, 3)

            if not data or len(data) < 3 or (data[0] == 0 and data[1] == 0):
                return False

            raw_val = (data[0] << 8) | data[1]
            if raw_val >= 32768: raw_val -= 65536
            otp.append(raw_val)

        self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]
        self.calibrated = True
        print(f"[ICP-10111] Ready! OTP Constants: C1={self.c1}, C2={self.c2}, C3={self.c3}, C4={self.c4}")
        return True
    except Exception as e:
        print("[ICP-10111] Init Error:", e)
        return False

```



```

def _calc_conversion_constants(self, p_Pa, p_LUT):

    denom_c = (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
               p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
               p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    if abs(denom_c) < 1e-6: return None, None, None

    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
         p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
         p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / denom_c

    denom_a = (p_LUT[0] - p_LUT[1])
    if abs(denom_a) < 1e-6: return None, None, None

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / denom_a
    B = (p_Pa[0] - A) * (p_LUT[0] + C)
    return A, B, C

def get_pressure(self):
    if not self.calibrated:
        if not self.init_sensor(): return None

    try:
        # Trigger Normal Mode Measurement (Command 0x6825)[cite: 1]
        self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))
        utime.sleep_ms(15) # Conversion delay

        res = self.i2c.readfrom(BARO_ADDRESS, 9)
        if not res or len(res) < 9: return None
        if (res[0] == 0 and res[1] == 0) or (res[0] == 0xFF and res[1] == 0xFF): return None

```

```
T_LSB = (res[0] << 8) | res[1]
p_LSB = (res[3] << 16) | (res[4] << 8) | res[6]
```

```
# OTP Compensation Math[cite: 1]
t = float(T_LSB - 32768)
s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])
if A is None or B is None or C is None: return None
```

```
denom_p = C + float(p_LSB)
if abs(denom_p) < 1e-6: return None
```

```
pressure_pa = A + (B / denom_p)
return pressure_pa
except:
    return None
```

```
icp_baro = ICP10111(i2c)
```

```
# -----
```

```
# LSM303DLHC SENSOR DRIVERS
```

```
# -----
```

```
def init_lsm303():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # Accel 100Hz ODR
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High Res
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # Mag 30Hz ODR
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous
```

```

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)

    if data is None: return None

    return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
    signed_12bit((data[5]<<8)|data[4])

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)

    if data is None: return None

    return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
    signed_16bit((data[4]<<8)|data[5])

# -----
# SENSOR CALIBRATION ROUTINE
# -----

def calibrate_system():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure

    print("\n=====")
    print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor STILL)")
    print("=====")
    sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0

    while count < 100:
        acc = read_accel()
        p = icp_baro.get_pressure()
        if acc and p:
            sx += acc[0]; sy += acc[1]; sz += acc[2]
            p_sum += p; count += 1
        utime.sleep_ms(20)

```

```

x_bias = sx / count
y_bias = sy / count
z_bias = (sz / count) - 1024.0 # Subtract gravity
baseline_pressure = p_sum / count

print(f"-> Accel Biases: X={x_bias:.1f}, Y={y_bias:.1f}, Z={z_bias:.1f}")
print(f"-> Baseline Ground Pressure: {baseline_pressure:.2f} Pa")

print("\n=====")
print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")
print("=====")
min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000:
    m = read_mag()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]
        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0
print(f"-> Mag Biases: X={mag_x_bias:.1f}, Y={mag_y_bias:.1f}")
print("=====\\n")

# -----
# NAVIGATION MATH
# -----

```

```

def get_heading():
    m = read_mag()

    if m is None: return 0.0

    mx_c = m[0] - mag_x_bias
    my_c = m[1] - mag_y_bias

    heading = math.degrees(math.atan2(my_c, mx_c))

    return heading + 360.0 if heading < 0 else heading


def get_cardinal(h):
    if h >= 337.5 or h < 22.5: return "N"
    elif 22.5 <= h < 67.5: return "NE"
    elif 67.5 <= h < 112.5: return "E"
    elif 112.5 <= h < 157.5: return "SE"
    elif 157.5 <= h < 202.5: return "S"
    elif 202.5 <= h < 247.5: return "SW"
    elif 247.5 <= h < 292.5: return "W"
    else: return "NW"


def get_relative_altitude(p_current):
    if p_current is None or baseline_pressure == 0: return 0.0

    return 44330.0 * (1.0 - math.pow((p_current / baseline_pressure), 0.1903))


# -----
# MAIN EXECUTION LOOP
# -----

def main():
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z
    global current_floor, last_heading

    init_lsm303()

    icp_baro.init_sensor()

```

```
calibrate_system()
```

```
smooth_accel = 0.0
```

```
accel_buffer = []
```

```
print(">>> STANDALONE 3D-PDR SYSTEM ONLINE: START WALKING <<<\n")
```

```
while True:
```

```
    now = utime.ticks_ms()
```

```
    acc = read_accel()
```

```
    if acc:
```

```
        ax = acc[0] - x_bias
```

```
        ay = acc[1] - y_bias
```

```
        az = acc[2] - z_bias
```

```
    # Magnitude & EMA Filter
```

```
    raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
```

```
    smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)
```

```
    accel_buffer.append(smooth_accel)
```

```
    if len(accel_buffer) > 15: accel_buffer.pop(0)
```

```
    if len(accel_buffer) == 15:
```

```
        avg_buffer = sum(accel_buffer) / 15.0
```

```
    # 1. Barometric Altitude & Floor Level Update (ICP-10111)
```

```
    current_p = icp_baro.get_pressure()
```

```
    if current_p is not None:
```

```
        pos_z = get_relative_altitude(current_p)
```

```
        current_floor = int(round(pos_z / FLOOR_HEIGHT_M))
```

2. Standalone Heuristic Step Detection

```
if (smooth_accel > ACCEL_THRESHOLD) and (smooth_accel > (avg_buffer * 1.40)):
```

```
    time_diff = utime.ticks_diff(now, last_step_time)
```

```
    if time_diff > STEP_COOLDOWN_MS:
```

```
        total_steps += 1
```

```
        total_distance += STRIDE_LENGTH_M
```

```
        last_step_time = now
```

```
        heading = get_heading()
```

```
        cardinal = get_cardinal(heading)
```

Turn Detection

```
heading_diff = abs(heading - last_heading)
```

```
if heading_diff > 180: heading_diff = 360 - heading_diff
```

```
if last_heading != 0 and heading_diff > 25:
```

```
    turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"
```

```
    print(f"  [!] TURN DETECTED: {turn} by {heading_diff:.1f}°")
```

```
    last_heading = heading
```

2D Position Trigonometry (X, Y)

```
rad = math.radians(heading)
```

```
pos_x += STRIDE_LENGTH_M * math.sin(rad)
```

```
pos_y += STRIDE_LENGTH_M * math.cos(rad)
```

Output Live Standalone Tracking Status

```
print(f"STEP #{total_steps} | Dir: {cardinal} ({heading:.1f}°) | "
```

```
    f"Dist: {total_distance:.1f}m | "
```

```
    f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "
```

```
    f"Height(Z): {pos_z:+.2f}m (Floor {current_floor})")
```

```

    utime.sleep_ms(20)

if __name__ == "__main__":
    main()

```

To achieve a strict separation between **real walking (arm swing / pendulum rhythm)** and **non-walking movements** (such as hand fidgeting, table taps, cable tugs, or sudden sensor pick-ups), relying on a single static threshold is not enough.

This complete, production-ready script uses **rolling statistical feature extraction (Standard Deviation, Peak-to-Peak Delta, and Moving Variance)** combined with **strict physical motion filters** to gate the step counter. It uses MicroPython's machine.I2C (**Hard I²C**) on your hardware and integrates the **LSM303DLHC** (Accel + Mag) and **ICP-10111** (Barometer) with zero-division error protection.

Key Non-Walk Filtering Features in This Code:

1. **Physical Motion State Gating:**
 - IDLE / STILL: Low variance ($\sigma \leq 1.85$). Step counting is frozen.
 - SHAKING / CABLE NOISE / TAPS: High peak-to-peak delta ($\Delta P > 82.5$) or sudden erratic spikes. Frozen.
 - WALKING: Smooth, rhythmic pendulum oscillations. Steps are **only** counted here.
2. **Hardware Hard I²C Bus:** Explicitly uses machine.I2C(0, ...) hardware peripheral blocks on the RT1064 EVK.
3. **Live Position & Location Status:** Displays real-time 3D Cartesian coordinates (X, Y, Z), compass heading, absolute motion state, and floor detection (Z -axis mapping via ICP-10111).

Complete MicroPython Program (main.py)

```

Python

# =====

# COMPLETE HARD I2C 3D INDOOR TRACKING & MOTION DISCRIMINATION PROGRAM

# Hardware: MIMXRT1064-EVK (Hard I2C), LSM303DLHC, ICP-10111 Barometer

# Features: Strict Walk vs Non-Walk Filter, 2D (X,Y) Coordinates, Floor Level (Z)

# =====

from machine import Pin, I2C

import utime, math

# --- HARDWARE I2C CONFIGURATION (HARD I2C) ---

```



```
I2C_ID = 0      # Hard I2C Bus 0
SDA_PIN = 0     # Hard SDA Pin
SCL_PIN = 1     # Hard SCL Pin
I2C_FREQ = 100000 # 100 kHz Standard Speed
```

```
# Target I2C Addresses
```

```
ACC_ADDRESS = 0x19
MAG_ADDRESS = 0x1E
BARO_ADDRESS = 0x63
```

```
# LSM303 Registers
```

```
CTRL_REG1_A = 0x20
CTRL_REG4_A = 0x23
OUT_X_L_A = 0x28
CRA_REG_M = 0x00
CRB_REG_M = 0x01
MR_REG_M = 0x02
OUT_X_H_M = 0x03
```

```
# Tuning & Physical Parameters
```

```
STRIDE_LENGTH_M = 0.50    # Distance per step in meters
STEP_COOLDOWN_MS = 380    # Minimum human gait step interval (ms)
ALPHA = 0.30              # EMA Low-pass filter coefficient
FLOOR_HEIGHT_M = 3.0      # Height per building floor in meters
```

```
# Global Calibration & Tracking Variables
```

```
x_bias, y_bias, z_bias = 0.0, 0.0, 0.0
mag_x_bias, mag_y_bias = 0.0, 0.0
baseline_pressure = 0.0
```

```
total_steps = 0
```

```

total_distance = 0.0

pos_x, pos_y, pos_z = 0.0, 0.0, 0.0

current_floor = 0

last_step_time = 0

last_heading = 0.0

current_motion_state = "IDLE/STILL"


# Initialize HARDWARE I2C Bus

i2c = I2C(I2C_ID, freqs=I2C_FREQ, sda=Pin(SDA_PIN), scl=Pin(SCL_PIN))


# -----

# LOW-LEVEL HARDWARE I2C DRIVERS

# -----

def write_reg(addr, reg, val):

    try: i2c.writeto_mem(addr, reg, bytes([val]))

    except: pass


def read_regs(addr, reg, length):

    try: return i2c.readfrom_mem(addr, reg, length)

    except: return None


def signed_12bit(val):

    val >>= 4

    if val >= 2048: val -= 4096

    return val


def signed_16bit(val):

    if val >= 32768: val -= 65536

    return val


# -----

```

```
# ICP-10111 BAROMETER DRIVER (With OTP Reading & Zero-Division Safety)
```

```
# -----
```

```
class ICP10111:
```

```
    def __init__(self, i2c_bus):
```

```
        self.i2c = i2c_bus
```

```
        self.c1, self.c2, self.c3, self.c4 = 0, 0, 0, 0
```

```
        self.calibrated = False
```

```
        self.p_Pa_calib = [45000.0, 80000.0, 105000.0]
```

```
        self.LUT_lower = 3.5 * (1 << 20)
```

```
        self.LUT_upper = 11.5 * (1 << 20)
```

```
        self.quadr_factor = 1.0 / 16777216.0
```

```
        self.offst_factor = 2048.0
```

```
    def init_sensor(self):
```

```
        try:
```

```
            # Soft Reset (0x805D)
```

```
            self.i2c.writeto(BARO_ADDRESS, bytes([0x80, 0x5D]))
```

```
            utime.sleep_ms(10)
```

```
            # Setup OTP Pointer (0xC59500669C)
```

```
            self.i2c.writeto(BARO_ADDRESS, bytes([0xC5, 0x95, 0x00, 0x66, 0x9C]))
```

```
            utime.sleep_ms(5)
```

```
            # Read 4 OTP Parameters[cite: 1]
```

```
            otp = []
```

```
            for _ in range(4):
```

```
                self.i2c.writeto(BARO_ADDRESS, bytes([0xC7, 0xF7]))
```

```
                data = self.i2c.readfrom(BARO_ADDRESS, 3)
```

```
                if not data or len(data) < 3 or (data[0] == 0 and data[1] == 0):
```

```
                    return False
```

```
                raw_val = (data[0] << 8) | data[1]
```

```

        if raw_val >= 32768: raw_val -= 65536

        otp.append(raw_val)

    self.c1, self.c2, self.c3, self.c4 = otp[0], otp[1], otp[2], otp[3]

    self.calibrated = True

    print(f"[ICP-10111 Hard I2C] Connected! OTP: C1={self.c1}, C2={self.c2}, C3={self.c3},
C4={self.c4}")

    return True

except Exception as e:

    print("[!] ICP-10111 Hard I2C Error:", e)

    return False


def _calc_conversion_constants(self, p_Pa, p_LUT):

    denom_c = (p_LUT[2] * (p_Pa[0] - p_Pa[1]) +
               p_LUT[0] * (p_Pa[1] - p_Pa[2]) +
               p_LUT[1] * (p_Pa[2] - p_Pa[0]))

    if abs(denom_c) < 1e-6: return None, None, None

    C = (p_LUT[0] * p_LUT[1] * (p_Pa[0] - p_Pa[1]) +
         p_LUT[1] * p_LUT[2] * (p_Pa[1] - p_Pa[2]) +
         p_LUT[2] * p_LUT[0] * (p_Pa[2] - p_Pa[0])) / denom_c

    denom_a = (p_LUT[0] - p_LUT[1])

    if abs(denom_a) < 1e-6: return None, None, None

    A = (p_Pa[0] * p_LUT[0] - p_Pa[1] * p_LUT[1] - (p_Pa[1] - p_Pa[0]) * C) / denom_a
    B = (p_Pa[0] - A) * (p_LUT[0] + C)

    return A, B, C


def get_pressure(self):

    if not self.calibrated:

```

```

if not self.init_sensor(): return None

try:
    # Trigger Normal Mode Measurement (Command 0x6825)[cite: 1]
    self.i2c.writeto(BARO_ADDRESS, bytes([0x68, 0x25]))
    utime.sleep_ms(15) # Conversion delay

    res = self.i2c.readfrom(BARO_ADDRESS, 9)
    if not res or len(res) < 9: return None
    if (res[0] == 0 and res[1] == 0) or (res[0] == 0xFF and res[1] == 0xFF): return None

    T_LSB = (res[0] << 8) | res[1]
    p_LSB = (res[3] << 16) | (res[4] << 8) | res[6]

    # OTP Compensation Math[cite: 1]
    t = float(T_LSB - 32768)
    s1 = self.LUT_lower + float(self.c1 * t * t) * self.quadr_factor
    s2 = self.offst_factor * self.c4 + float(self.c2 * t * t) * self.quadr_factor
    s3 = self.LUT_upper + float(self.c3 * t * t) * self.quadr_factor

    A, B, C = self._calc_conversion_constants(self.p_Pa_calib, [s1, s2, s3])
    if A is None or B is None or C is None: return None

    denom_p = C + float(p_LSB)
    if abs(denom_p) < 1e-6: return None

    pressure_pa = A + (B / denom_p)
    return pressure_pa
except:
    return None

```

```

icp_baro = ICP10111(i2c)

# -----

# LSM303DLHC DRIVERS

# -----

def init_lsm303():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # Accel 100Hz ODR
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High Res
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # Mag 30Hz ODR
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | 0x80, 6)
    if data is None: return None
    return signed_12bit((data[1]<<8)|data[0]), signed_12bit((data[3]<<8)|data[2]),
    signed_12bit((data[5]<<8)|data[4])

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None: return None
    return signed_16bit((data[0]<<8)|data[1]), signed_16bit((data[2]<<8)|data[3]),
    signed_16bit((data[4]<<8)|data[5])

# -----

# CALIBRATION ROUTINE

# -----

def calibrate_system():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias, baseline_pressure

    print("\n=====")
    print("STEP 1/2: ACCEL & BARO CALIBRATION (Keep sensor STILL)")

```

```

print("=====")

sx, sy, sz, p_sum, count = 0, 0, 0, 0, 0

while count < 100:
    acc = read_accel()
    p = icp_baro.get_pressure()
    if acc and p:
        sx += acc[0]; sy += acc[1]; sz += acc[2]
        p_sum += p; count += 1
    utime.sleep_ms(20)

x_bias = sx / count
y_bias = sy / count
z_bias = (sz / count) - 1024.0 # Subtract gravity vector
baseline_pressure = p_sum / count

print(f"-> Accel Biases: X={x_bias:.1f}, Y={y_bias:.1f}, Z={z_bias:.1f}")
print(f"-> Baseline Ground Pressure: {baseline_pressure:.2f} Pa")

print("\n=====")
print("STEP 2/2: MAGNETOMETER CALIBRATION (Rotate in Figure-8)")
print("=====")
min_x, max_x, min_y, max_y = 32767, -32768, 32767, -32768

start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 7000:
    m = read_mag()
    if m:
        if m[0] < min_x: min_x = m[0]
        if m[0] > max_x: max_x = m[0]
        if m[1] < min_y: min_y = m[1]

```

```

        if m[1] > max_y: max_y = m[1]
    utime.sleep_ms(20)

mag_x_bias = (max_x + min_x) / 2.0
mag_y_bias = (max_y + min_y) / 2.0
print(f"-> Mag Biases: X={mag_x_bias:.1f}, Y={mag_y_bias:.1f}")
print("=====\n")

# -----
# STRICT WALK vs NON-WALK MOTION CLASSIFIER ENGINE
# -----

def classify_strict_motion(accel, avg, std, p2p):
    """
    Evaluates dynamic features over 15 rolling samples:
    Class 0: IDLE / STILL (Stationary or minimal hand drift)
    Class 1: REAL WALKING (Rhythmic pendulum swing signature)
    Class 2: NON-WALKING NOISE (Cable tugs, shaking, table taps, fidgeting)
    """
    if std <= 1.85:
        return 0, "IDLE / STILL"
    elif p2p > 82.5 or std > 14.0:
        return 2, "REJECTED NOISE (SHAKING/TAP)"
    else:
        # Check for rhythmic arm swing pendulum pulse
        if accel > (avg * 1.15) and accel < (avg * 1.85):
            return 1, "REAL WALKING"
        else:
            return 0, "SWING RECOVERY"

# -----
# NAVIGATION & ELEVATION MATH

```



```

# -----

def get_heading():
    m = read_mag()

    if m is None: return 0.0

    mx_c = m[0] - mag_x_bias
    my_c = m[1] - mag_y_bias

    heading = math.degrees(math.atan2(my_c, mx_c))

    return heading + 360.0 if heading < 0 else heading


def get_cardinal(h):
    if h >= 337.5 or h < 22.5: return "NORTH"
    elif 22.5 <= h < 67.5: return "NORTH-EAST"
    elif 67.5 <= h < 112.5: return "EAST"
    elif 112.5 <= h < 157.5: return "SOUTH-EAST"
    elif 157.5 <= h < 202.5: return "SOUTH"
    elif 202.5 <= h < 247.5: return "SOUTH-WEST"
    elif 247.5 <= h < 292.5: return "WEST"
    else: return "NORTH-WEST"


def get_relative_altitude(p_current):
    if p_current is None or baseline_pressure == 0: return 0.0

    return 44330.0 * (1.0 - math.pow((p_current / baseline_pressure), 0.1903))


# -----

# MAIN EXECUTION LOOP

# -----

def main():
    global total_steps, total_distance, last_step_time, pos_x, pos_y, pos_z

    global current_floor, last_heading, current_motion_state

    init_lsm303()

```

```

icp_baro.init_sensor()

calibrate_system()


smooth_accel = 0.0
accel_buffer = []


print(">>> 3D TRACKING ONLINE: STRICT WALK vs NON-WALK FILTER ACTIVE <<<\n")


while True:

    now = utime.ticks_ms()

    acc = read_accel()


    if acc:

        ax = acc[0] - x_bias
        ay = acc[1] - y_bias
        az = acc[2] - z_bias


        # Magnitude & EMA Filter
        raw_mag = math.sqrt(ax**2 + ay**2 + az**2)
        smooth_accel = (ALPHA * raw_mag) + ((1.0 - ALPHA) * smooth_accel)


        accel_buffer.append(smooth_accel)

        if len(accel_buffer) > 15: accel_buffer.pop(0)


        if len(accel_buffer) == 15:

            # 1. Feature Extraction Across Rolling Window
            avg_val = sum(accel_buffer) / 15.0
            variance = sum((v - avg_val)**2 for v in accel_buffer) / 15.0
            std_dev = math.sqrt(variance)
            p2p_delta = max(accel_buffer) - min(accel_buffer)

```

2. Strict Motion Classification

```
motion_code, motion_label = classify_strict_motion(smooth_accel, avg_val, std_dev,
p2p_delta)
```

```
if motion_label != current_motion_state:
    print(f"\n[STATE CHANGE]: {motion_label}")
    current_motion_state = motion_label
```

3. Read ICP-10111 Altitude & Detect Floor Index

```
current_p = icp_baro.get_pressure()
if current_p is not None:
    pos_z = get_relative_altitude(current_p)
    current_floor = int(round(pos_z / FLOOR_HEIGHT_M))
```

4. Step Execution Gating (STRICT: ONLY Code 1 = REAL WALKING)

```
if motion_code == 1:
    time_diff = utime.ticks_diff(now, last_step_time)
    if time_diff > STEP_COOLDOWN_MS:
        total_steps += 1
        total_distance += STRIDE_LENGTH_M
        last_step_time = now
```

```
heading = get_heading()
cardinal = get_cardinal(heading)
```

Turn Detection

```
heading_diff = abs(heading - last_heading)
if heading_diff > 180: heading_diff = 360 - heading_diff
if last_heading != 0 and heading_diff > 25:
    turn = "RIGHT" if (heading - last_heading) % 360 < 180 else "LEFT"
    print(f" [!] TURN DETECTED: {turn} by {heading_diff:.1f}°")
```

```
last_heading = heading
```

```
# 2D Position Trigonometry (X, Y)
```

```
rad = math.radians(heading)
```

```
pos_x += STRIDE_LENGTH_M * math.sin(rad)
```

```
pos_y += STRIDE_LENGTH_M * math.cos(rad)
```

```
# Output Live Complete Location & Tracking Status
```

```
print(f"STEP #{total_steps:03d} | "
```

```
    f"Motion: {motion_label} | "
```

```
    f"Facing: {cardinal} ({heading:.1f}°) | "
```

```
    f"Dist: {total_distance:.1f}m | "
```

```
    f"Pos(X,Y): ({pos_x:.2f}m, {pos_y:.2f}m) | "
```

```
    f"Height(Z): {pos_z:+.2f}m | "
```

```
    f"Floor: {current_floor}")
```

```
    utime.sleep_ms(20)
```

```
if __name__ == "__main__":
```

```
    main()
```